Manual for running the design on AWS EC2 F2 instance

Introduction
- What is this page?
- Key Concepts
- The High-Level Workflow

Step-by-step Tutorial
- 0. Video Walkthrough
- 1. Create an EC2 instance
- 2. Ubuntu GUI Setup
- 3. Generate AFI from design
- 4. Run design on F2
- 5. All AFIs created so far

Introduction

▼ What is this page?

This manual documents the process of taking our RTL design (originally targeted for local hardware like Opal Kelly or Nexys) and migrating it to run on **AWS EC2 F2 instances**.

Unlike a traditional FPGA workflow where you generate a bitstream locally and flash it to a board on your desk, the AWS workflow is cloud-based and involves specific proprietary steps to ensure security and compatibility with the data center infrastructure.

▼ Key Concepts

Before diving into the steps, you must understand three AWS-specific terms:

- **CL (Custom Logic):** This is **your design** (your RTL). In the AWS ecosystem, you do not own the entire FPGA. You own a specific region called the "Custom Logic."
- **Shell (SH):** The "wrapper" provided by AWS that surrounds your CL. The Shell handles all the physical I/O (PCIe, DRAM, Ethernet) and exposes standard interfaces (like AXI4) to your design. You cannot touch the physical pins directly; you must follow the **CL Protocols** to communicate with the Shell.
  - for more see details about our shell implementation, refer to the section below
- **AFI (Amazon FPGA Image):** Think of this as the **bitstream**. You cannot download a raw `.bit` file to an AWS FPGA. Instead, you generate a Design Checkpoint (DCP), send it to AWS to be encrypted and formatted, and they give you back an **AFI ID** (e.g., `afi-0xxxx...`). You use this ID to load your design onto the instance.

▼ The High-Level Workflow

The process to get running on F2 follows this cycle:

1. **Design & Simulation:** Write your RTL (SystemVerilog/Verilog) and verify it. You must wrap your top-level module to match the AWS Shell interface (AXI-Lite for register access, AXI4 for bulk data).

2. **Build (The "Tarball"):** Use the AWS HDK (Hardware Dev Kit) to compile your design into a specific format. This step generates a design checkpoint or tarball.
3. **Create AFI:** Submit your tarball to AWS. This starts a background process where AWS converts your design into an **AFI**. This can take time.
4. **Load & Run:** Once the AFI is "Available," you use the FPGA Management Tools on the F2 instance to load the AFI ID into the FPGA slot.
5. **Host Software:** Run your C++/Python software on the F2 instance's CPU to interact with the FPGA via PCIe.

Step-by-step Tutorial

▼ 0. Video Walkthrough

- A video is made which roughly includes:
  - create an instance with FPGA AMI
  - getting the Ubuntu GUI
  - cloning repo and design files walkthrough
  - generating DCP & AFI
  - launching the design on FPGA & GUI
- https://drive.google.com/file/d/1dAEO2YmVMeRsG_TutIWvumbxLFbet8KS/view?usp=sharing

▼ 1. Create an EC2 instance

- register for AWS account, make sure to get the free tier and $120 credit
- go to AWS EC2 console, make sure in the correct region
  - e.g. F2 instance only exists in us-east-1
  - for instance without FPGA, normally ca-central-1 is good
- can use a cheaper instance for building the image
  - e.g. m7i-flex.large
- for the F2 instance, need to first create a ticket to get access to this instance
  - ask AI for instructions, the review process takes ~2 days
- for the AMI, load FPGA Developer AMI (Ubuntu) Version 1.19.0
- for security group, key, ssh... ask AI

▼ 2. Ubuntu GUI Setup

- method 1 - manual:
  - main guide: https://awsdocs-fpga-f2.readthedocs-hosted.com/latest/developer-resources/Amazon-DCV-Setup-Guide.html
  - first time setup

        - follow all steps in the main guide
        - note: use Ubuntu 22.04 rather than 20.04
- method 2 - single helper script
    - https://repost.aws/articles/ARJtZxRiOURwWI2qSWjI4AaQ/install-gui-graphical-desktop-on-amazon-ec2-instances-running-ubuntu-linux
    - in the middle there's a huge script that starts with `#!/bin/bash`
    - before running
        - `echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf`
        - `echo "127.0.1.1 $(hostname)" | sudo tee -a /etc/hosts`
        - in security group add port 8443 for inbound TCP rule
    - fixed script that works for AMI v1.19.0

```
#!/bin/bash
export DEBIAN_FRONTEND=noninteractive
sudo apt update
sudo apt install -y ubuntu-desktop-minimal
sudo sed -i '/^\[daemon\]/a WaylandEnable=false' /etc/gdm3/custom.conf
sudo systemctl set-default graphical.target

cd /tmp
# Direct link for Ubuntu 24.04 (Noble) - Version 2024.0
wget https://d1uj6qtbmh3dt5.cloudfront.net/2024.0/Servers/nice-dcv-202
4.0-17979-ubuntu2404-x86_64.tgz
tar -xvzf nice-dcv-2024.0-17979-ubuntu2404-x86_64.tgz
cd nice-dcv-2024.0-17979-ubuntu2404-x86_64

# Install the specific deb files
sudo apt install -y ./nice-dcv-server_*.deb ./nice-dcv-web-viewer_*.deb
./nice-xdcv_*.deb
sudo usermod -aG video dcv
sudo systemctl enable dcvserver

# Configuration
sudo sed -i "/^\[session-management\/automatic-console-session/a owner=
\"ubuntu\"\nstorage-root=\"%home%\"" /etc/dcv/dcv.conf
sudo sed -i "s/^#create-session/create-session/g" /etc/dcv/dcv.conf

# Setup dummy X server for headless EC2
sudo apt install -y xserver-xorg-video-dummy
sudo tee /etc/X11/xorg.conf > /dev/null << EOF
Section "Device"
    Identifier "DummyDevice"
```

```
    Driver "dummy"
    VideoRam 512000
EndSection

Section "Monitor"
    Identifier "DummyMonitor"
    HorizSync   5.0 - 1000.0
    VertRefresh 5.0 - 200.0
EndSection

Section "Screen"
    Identifier "DummyScreen"
    Device "DummyDevice"
    Monitor "DummyMonitor"
    DefaultDepth 24
    SubSection "Display"
        Viewport 0 0
        Depth 24
        Virtual 4096 2160
    EndSubSection
EndSection
EOF

# Restart and Start
sudo systemctl isolate multi-user.target && sudo systemctl isolate grap
hical.target
sudo systemctl stop dcvserver && sudo systemctl start dcvserver

# Set your password for the GUI login
echo "Please set a password for the 'ubuntu' user to log in via DCV:"
sudo passwd ubuntu
```

- GUI might stuck for first time use, if that's the case, force stop the instance and restart
- every time starting an instance
  - `sudo systemctl restart dcvserver.service`
  - `sudo systemctl restart dcvsessionlauncher.service`
  - `dcv list-sessions`
    - nothing should appear
    - if yes, delete it - `sudo dcv close-session <name>`
  - `dcv create-session 1`
  - `dcv list-sessions`
    - 1 should appear

- in browser, enter `https://<public_dns>/#1`
  - public_dns can be found in EC2 console
  - e.g. `ec2-35-182-196-197.ca-central-1.compute.amazonaws.com:8443`

▼ 3. Generate AFI from design

- Main guide: https://awsdocs-fpga-f2.readthedocs-hosted.com/latest/hdk/README.html
- Required package
  - `git clone https://github.com/aws/aws-fpga.git`
- Design files
  - `git clone https://github.com/elizabethtyra/ece496-capstone.git`
  - copy the `cl_sys6` folder under `aws-fpga/hdk/cl/examples/`
  - make sure the design/ folder does not contain subfolders
- Build
  - cd to the aws-fpga folder
  - `source /etc/profile.d/default_module.sh`
  - `cd ~/capstone/aws-fpga`
  - `source hdk_setup.sh`
  - `cd hdk/cl/examples/cl_sys6/`
  - `export CL_DIR=$(pwd)`
  - `cd build/scripts`
  - `./aws_build_dcp_from_cl.py -c cl_sys6 --mode small_shell --aws_clk_gen --clock_recipe_a A2`
- Create AFI
  - wrapper for everything below
    - `cd ~/capstone/aws-fpga/hdk/cl/examples/cl_sys6`
    - usage: `sh dcp2afi.sh <bucket_folder_name> <cl_dcp.tar>`
    - example: `sh dcp2afi.sh sys6_pam6_1core_62.5M_20260106 /home/ubuntu/capstone/aws-fpga/hdk/cl/examples/cl_sys6/build/checkpoints/2026_01_07-011554.Developer_CL.tar`
  - `export DCP_BUCKET_NAME='uoft-capstone-2025148-bucket'; export LOGS_BUCKET_NAME='uoft-capstone-2025148-log-bucket'; export REGION='us-east-1'`
  - `export DCP_FOLDER_NAME='sys6_pam6_1core_62.5M_20260106'`
    - change every time
  - `export LOGS_FOLDER_NAME='sys6_pam6_1core_62.5M_20260106_logs'`
    - change every time
  - `export DCP_TARBALL_TO_INGEST='/home/ubuntu/capstone/aws-fpga/hdk/cl/examples/cl_sys6/build/checkpoints/2026_01_07-011554.Developer_CL.tar'`
    - change every time

- `aws s3 mb s3://${DCP_BUCKET_NAME} --region ${REGION}`
- `aws s3 mb s3://${DCP_BUCKET_NAME}/${DCP_FOLDER_NAME}/`
- `aws s3 cp ${DCP_TARBALL_TO_INGEST} s3://${DCP_BUCKET_NAME}/${DCP_FOLDER_NAME}/`
- `aws s3 mb s3://${LOGS_BUCKET_NAME}/${LOGS_FOLDER_NAME}/ --region ${REGION}`
- `touch LOGS_FILES_GO_HERE.txt`
- `aws s3 cp LOGS_FILES_GO_HERE.txt s3://${LOGS_BUCKET_NAME}/${LOGS_FOLDER_NAME}/`
- `export DCP_TARBALL_NAME=$(basename ${DCP_TARBALL_TO_INGEST})`
- `export CL_DESIGN_NAME='cl_sys6'`
- `export CL_DESIGN_DESCRIPTION="Description of ${CL_DESIGN_NAME}"`
- `aws ec2 create-fpga-image --name ${CL_DESIGN_NAME} --description "${CL_DESIGN_DESCRIPTION}" --input-storage-location Bucket=${DCP_BUCKET_NAME},Key=${DCP_FOLDER_NAME}/${DCP_TARBALL_NAME} --logs-storage-location Bucket=${LOGS_BUCKET_NAME},Key=${LOGS_FOLDER_NAME}/ --region ${REGION}`

- Check AFI
  - after the previous step, two IDs will show up： afi-XX and agfi-XX

```
Creating FPGA image...
{
    "FpgaImageId": "afi-0babb361845f29993",
    "FpgaImageGlobalId": "agfi-04e41a1087522ee26"
}
Done!
```

  - `aws ec2 describe-fpga-images --region us-east-1 --fpga-image-ids afi-02a897cda37e73917`
    - change every time
    - wait the status to become “available” from “pending”

```
ubuntu@ip-172-31-20-211:~/capstone/aws-fpga/hdk/cl/examples/cl_sys6$ aws ec2 d
escribe-fpga-images --region us-east-1 --fpga-image-ids afi-0babb361845f29993

{
    "FpgaImages": [
        {
            "FpgaImageId": "afi-0babb361845f29993",
            "FpgaImageGlobalId": "agfi-04e41a1087522ee26",
            "Name": "cl_sys6",
            "Description": "Description of cl_sys6",
            "State": {
                "Code": "pending"
            },
            "CreateTime": "2026-01-24T22:01:15+00:00",
            "UpdateTime": "2026-01-24T22:01:15+00:00",
            "OwnerId": "198146919018",
            "Tags": [],
            "Public": false,
            "DataRetentionSupport": false
        }
    ]
}
```

        - now we are ready to move on to a F2 instance to run the design
- To give another one AFI access permission
    - in the machine that generated AFI
    - `aws ec2 modify-fpga-image-attribute --fpga-image-id afi-0c6305638492339e4 --operation-type add --user-ids 331005567917`

▼ 4. Run design on F2

- Create a F2 instance, using the FPGA AMI (see above)
- Clone aws_fpga repo and our git repo (see above)
- some libraries are required to get the GUI working
    - `sudo apt install -y python3-tk`
    - `sudo pip3 install customtkinter --break-system-packages`
    - `sudo apt install python3-matplotlib`
- Load AFI on F2
    - `cd aws-fpga; source sdk_setup.sh; sudo fpga-clear-local-image -S 0`
    - `sudo fpga-load-local-image -S 0 -I agfi-02af6cfffc88089a5`
        - note: use `sudo fpga-describe-local-image -S 0 -H` to check the loaded image
- Run controller software
    - runtime files are located in `GIT_REPO/sysAll_PAM6/runtime/`
    - `run.py` - non-GUI version
    - `noiseGen.py` - generates `noiseXXdB_sep24.mem`
    - `gui.py` - GUI that allows sweeping and auto creation of missing .mem files

▼ 5. All AFIs created so far

SYS	PAM	N_CORE	AFI	AGFI	NOTES
6	6	1	`afi-02a897cda37e73917`	`agfi-0ca6e4bd4b1fc25a0`	first working version, one core, stuck at 125MHz, noise not added
6	6	2	`afi-0c5db9e59bec0bd9b`	`agfi-069ee0508d2d9d72a`	with noise
6	6	2	`afi-0c6305638492339e4`	`agfi-0ae6d28929ea77ff7`	`total_frames` and `total_frame_errs` added
6	4	2	`afi-0f0946631a934304f`	`agfi-0688e763b7fb3e61e`	used for platform validation
5	4	2	`afi-088b2bf689c7f1349`	`agfi-0c910eb1d54d5026f`	
5	6	1	`afi-01386880913961b1d`	`agfi-0b1a7cdf5fb4e4b59`	

6	6	10	`afi-0b68f60addadf15a6`	`agfi-02af6cfffc88089a5`	used for final demo
4	4	2	`afi-0a14db5b9c0d96946`	`agfi-0b9598a87f66c89ba`	
4	6	1	`afi-0032baec4aed76c0b`	`agfi-0484509c16f950518`	